MODBUS Protocol Specification for AI Implementation

This document provides a comprehensive technical reference for the MODBUS protocol, optimized for AI agents generating MODBUS client/server implementations. It prioritizes precision, byte-level specifications, and concrete examples.

---

1. Overview

MODBUS is an application-layer messaging protocol (OSI Layer 7) providing client/server communication between devices. Originally developed by Modicon in 1979 for programmable logic controllers, it remains the de facto standard for industrial device communication.

Official Resources

Resource	URL
MODBUS Organization	https://www.modbus.org/
Official Specifications	https://www.modbus.org/modbus-specifications
FAQ & History	https://www.modbus.org/faq

Key Characteristics

Property	Value
Architecture	Client/Server (Master/Slave)
Data encoding	Big-endian
Max PDU size	253 bytes
TCP port	502 (standard), 802 (secure/TLS)
Byte order	Most significant byte first

Transport Variants

Variant	Medium	Error Check	Framing
MODBUS TCP	Ethernet	TCP/IP	MBAP header (7 bytes)
MODBUS RTU	Serial (RS-485/232)	CRC-16	Silent intervals

Variant	Medium	Error Check	Framing
MODBUS ASCII	Serial	LRC	Start ':' / End CR-LF
MODBUS TCP Security	Ethernet + TLS	TCP/IP + TLS	MBAP over TLS

---

2. Data Model

MODBUS defines four primary data tables. Devices may implement these as separate memory areas or as overlapping views of the same memory.

Primary Tables

Table	Object Type	Size	Access	Typical Use
Coils	Bit	1 bit	Read/Write	Digital outputs, relays
Discrete Inputs	Bit	1 bit	Read-only	Digital inputs, switches
Holding Registers	Word	16 bits	Read/Write	Configuration, setpoints
Input Registers	Word	16 bits	Read-only	Measurements, status

Addressing

CRITICAL: PDU addresses are 0-based. Documentation often uses 1-based numbering.

Data Model Reference	PDU Address	Wire Value
Register 1	0	0x0000
Register 100	99	0x0063
Register 40001	0	0x0000 (Holding Register context)

The "40001" notation is a legacy convention where:

- 0xxxx = Coils
- 1xxxx = Discrete Inputs
- 3xxxx = Input Registers
- 4xxxx = Holding Registers

For implementation: Always use 0-based addresses in PDUs.

Data Encoding

All multi-byte values use **big-endian** (network byte order):

```
Value: 0x1234
Transmitted: [0x12] [0x34]
             MSB    LSB
```

For 32-bit values across two registers, common conventions:

- **Big-endian (default)**: High word first → Register N = MSW, Register N+1 = LSW
- **Little-endian (some devices)**: Low word first → Register N = LSW, Register N+1 = MSW

Always verify device documentation for 32-bit value ordering.

---

3. Protocol Data Unit (PDU)

The PDU is transport-independent and has a maximum size of 253 bytes.

Request PDU

```
[Function Code: 1 byte][Request Data: 0-252 bytes]
```

Response PDU

```
[Function Code: 1 byte][Response Data: 0-252 bytes]
```

Exception Response PDU

```
[Function Code + 0x80: 1 byte][Exception Code: 1 byte]
```

When an error occurs, the server returns the function code with bit 7 set (value + 0x80).

---

4. Transport Layers

4.1 MODBUS TCP (Primary)

MODBUS TCP encapsulates the PDU in a 7-byte MBAP (MODBUS Application Protocol) header.

MBAP Header Structure

Offset	Field	Size	Description
0	Transaction ID	2 bytes	Request/response matching (client-assigned)
2	Protocol ID	2 bytes	0x0000 for MODBUS
4	Length	2 bytes	Bytes following (Unit ID + PDU length)
6	Unit ID	1 byte	Slave address (0xFF for direct, 1-247 for gateway)

Complete TCP ADU

```
[Transaction ID: 2][Protocol ID: 2][Length: 2][Unit ID: 1][Function: 1]
[Data: N]
|<---------------- MBAP Header (7 bytes) --------------->|<----- PDU -------
>|
```

Maximum TCP ADU: 260 bytes (7 byte header + 253 byte PDU)

TCP ADU Example: Read Holding Registers

Read 10 registers starting at address 0 from Unit ID 1:

Request (12 bytes):

```
Transaction ID:  00 01
Protocol ID:     00 00
Length:          00 06  (6 bytes follow: Unit ID + PDU)
Unit ID:         01
Function Code:   03     (Read Holding Registers)
Start Address:   00 00  (Address 0)
Quantity:        00 0A  (10 registers)
```

Response (27 bytes):

```
Transaction ID:  00 01  (echoed)
Protocol ID:     00 00
Length:          00 15  (21 bytes follow)
Unit ID:         01
Function Code:   03
```

```
Byte Count:      14     (20 bytes = 10 registers × 2)
Register 0:      00 64  (value 100)
Register 1:      00 C8  (value 200)
... (8 more registers)
```

TCP Connection Guidelines

Guideline	Recommendation
Port	502 (server listens)
Connection lifetime	Keep open; don't reconnect per request
Concurrent requests	Supported; use unique Transaction IDs
Socket options	Enable TCP_NODELAY, SO_KEEPALIVE
Requests per TCP frame	One ADU per frame (don't batch)
Response timeout	Application-specific (typically 1-5 seconds)

4.2 MODBUS RTU (Serial)

RTU (Remote Terminal Unit) mode transmits data as binary with CRC-16 error checking.

RTU Frame Structure

```
[Address: 1][Function: 1][Data: N][CRC: 2]
|<- Slave ->|<------ PDU ------->|<-CRC->|
```

Maximum RTU ADU: 256 bytes (1 + 253 + 2)

Frame Timing

Frames are delimited by silent intervals (no characters transmitted):

Timing	Duration	Purpose
Inter-frame gap	≥ 3.5 character times	Marks frame boundaries
Inter-character max	< 1.5 character times	Characters within frame

At 9600 baud with 11 bits/character (1 start + 8 data + 1 parity + 1 stop):

- Character time = 11/9600 = 1.146 ms
- 1.5 char times = 1.72 ms

- 3.5 char times = 4.01 ms

CRC-16 Calculation

Parameter	Value
Polynomial	0x8005 (bit-reversed: 0xA001)
Initial value	0xFFFF
Input reflection	Yes (LSB first)
Output reflection	Yes
Final XOR	0x0000

CRC byte order: LSB first (little-endian)

```
Frame data: [01] [03] [00] [00] [00] [0A]
CRC-16:     0xC5CD
Transmitted: [01] [03] [00] [00] [00] [0A] [CD] [C5]
                                           LSB  MSB
```

CRC-16 Algorithm (pseudocode):

```
function crc16_modbus(data: bytes) -> uint16:
    crc = 0xFFFF
    for byte in data:
        crc = crc XOR byte
        for i in 0..7:
            if (crc AND 0x0001) != 0:
                crc = (crc >> 1) XOR 0xA001
            else:
                crc = crc >> 1
    return crc  // Return as-is; transmit LSB first
```

4.3 MODBUS ASCII (Serial)

ASCII mode transmits each byte as two ASCII hexadecimal characters.

ASCII Frame Structure

```
[':'][Address: 2 chars][Function: 2 chars][Data: 2N chars][LRC: 2 chars][CR]
[LF]
```

Field	Value
Start	':' (0x3A)
End	CR LF (0x0D 0x0A)
Data encoding	Each byte → 2 ASCII hex chars ('0'-'9', 'A'-'F')

LRC Calculation

LRC (Longitudinal Redundancy Check) is the two's complement of the 8-bit sum of all bytes (address through last data byte).

```
function lrc(data: bytes) -> uint8:
    sum = 0
    for byte in data:
        sum = (sum + byte) AND 0xFF
    return ((-sum) AND 0xFF)
```

Example:

```
Binary frame: [01] [03] [00] [00] [00] [0A]
Sum: 01 + 03 + 00 + 00 + 00 + 0A = 0x0E
LRC: -0x0E = 0xF2

ASCII frame: :0103000000F2<CR><LF>
             | |  |    | |
             | |  |    | +-- LRC as "F2"
             | |  |    +---- Data as "00000A"
             | |  +--------- Function as "03"
             | +------------ Address as "01"
             +-------------- Start character
```

4.4 MODBUS TCP Security (mbaps)

Secure MODBUS encapsulates standard MBAP/PDU within TLS.

Property	Value
Port	802
TLS version	1.2 minimum (1.3 recommended)
Authentication	Mutual TLS (client and server certificates)
Certificate format	x.509v3

Required Cipher Suite

```
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
```

Role-Based Authorization

Roles can be encoded in x.509v3 certificate extensions:

Property	Value
OID	1.3.6.1.4.1.50316.802.1
Encoding	ASN1 UTF8String
Usage	Server extracts role, applies authorization rules

Unauthorized requests receive exception code 0x01 (Illegal Function).

---

5. Function Codes

5.1 Summary Table

Code	Hex	Name	Access	Max Quantity
01	0x01	Read Coils	Coils	2000 bits
02	0x02	Read Discrete Inputs	Discrete Inputs	2000 bits
03	0x03	Read Holding Registers	Holding Registers	125 registers
04	0x04	Read Input Registers	Input Registers	125 registers
05	0x05	Write Single Coil	Coils	1 bit
06	0x06	Write Single Register	Holding Registers	1 register
15	0x0F	Write Multiple Coils	Coils	1968 bits
16	0x10	Write Multiple Registers	Holding Registers	123 registers
22	0x16	Mask Write Register	Holding Registers	1 register
23	0x17	Read/Write Multiple Registers	Holding Registers	R:125, W:121
43	0x2B	Encapsulated Interface Transport	Various	-

5.2 Function Code 0x03: Read Holding Registers

Most commonly used function code for reading device data.

Request (5 bytes)

Offset	Field	Size	Value Range
0	Function Code	1	0x03
1-2	Starting Address	2	0x0000 - 0xFFFF
3-4	Quantity of Registers	2	0x0001 - 0x007D (1-125)

Response (2 + 2N bytes)

Offset	Field	Size	Description
0	Function Code	1	0x03
1	Byte Count	1	2 × Quantity
2+	Register Values	2N	N registers, 2 bytes each

Example: Read 3 registers starting at address 107

Request PDU:

```
03 00 6B 00 03
|  |     |
|  |     +-- Quantity: 3 registers
|  +-------- Start address: 107 (0x006B)
+----------- Function code: 0x03
```

Response PDU:

```
03 06 02 2B 00 00 00 64
|  |  |     |     |
|  |  |     |     +-- Register 109: 0x0064 (100)
|  |  |     +-------- Register 108: 0x0000 (0)
|  |  +-------------- Register 107: 0x022B (555)
|  +---------------- Byte count: 6 bytes
+-------------------- Function code: 0x03
```

5.3 Function Code 0x04: Read Input Registers

Identical format to 0x03, but reads from Input Registers table.

Request (5 bytes)

Offset	Field	Size	Value Range
0	Function Code	1	0x04
1-2	Starting Address	2	0x0000 - 0xFFFF
3-4	Quantity of Registers	2	0x0001 - 0x007D (1-125)

Response (2 + 2N bytes)

Offset	Field	Size	Description
0	Function Code	1	0x04
1	Byte Count	1	2 × Quantity
2+	Register Values	2N	N registers, 2 bytes each

5.4 Function Code 0x06: Write Single Register

Request (5 bytes)

Offset	Field	Size	Value Range
0	Function Code	1	0x06
1-2	Register Address	2	0x0000 - 0xFFFF
3-4	Register Value	2	0x0000 - 0xFFFF

Response (5 bytes)

Echo of request (confirms write).

Offset	Field	Size	Description
0	Function Code	1	0x06
1-2	Register Address	2	Address written
3-4	Register Value	2	Value written

Example: Write 0x0003 to register 1

Request PDU:

```
06 00 01 00 03
|  |     |
|  |     +-- Value: 0x0003
|  +-------- Address: 1 (0x0001)
+----------- Function code: 0x06
```

Response PDU:

```
06 00 01 00 03  (echo of request)
```

5.5 Function Code 0x10: Write Multiple Registers

Request (6 + 2N bytes)

Offset	Field	Size	Value Range
0	Function Code	1	0x10
1-2	Starting Address	2	0x0000 - 0xFFFF
3-4	Quantity of Registers	2	0x0001 - 0x007B (1-123)
5	Byte Count	1	2 × Quantity
6+	Register Values	2N	Values to write

Response (5 bytes)

Offset	Field	Size	Description
0	Function Code	1	0x10
1-2	Starting Address	2	Address of first register
3-4	Quantity of Registers	2	Number of registers written

Example: Write 2 registers starting at address 1

Write 0x000A to register 1 and 0x0102 to register 2.

Request PDU:

```
10 00 01 00 02 04 00 0A 01 02
|  |     |     |  |     |
|  |     |     |  |     +-- Register 2 value: 0x0102
|  |     |     |  +-------- Register 1 value: 0x000A
```

```
|  |     |     +----------- Byte count: 4
|  |     +----------------- Quantity: 2 registers
|  +----------------------- Start address: 1
+--------------------------- Function code: 0x10
```

Response PDU:

```
10 00 01 00 02
|  |     |
|  |     +-- Quantity written: 2
|  +-------- Start address: 1
+----------- Function code: 0x10
```

5.6 Function Code 0x01: Read Coils

Request (5 bytes)

Offset	Field	Size	Value Range
0	Function Code	1	0x01
1-2	Starting Address	2	0x0000 - 0xFFFF
3-4	Quantity of Coils	2	0x0001 - 0x07D0 (1-2000)

Response (2 + N bytes)

Offset	Field	Size	Description
0	Function Code	1	0x01
1	Byte Count	1	⌈Quantity / 8⌉
2+	Coil Status	N	Packed bits, LSB = lowest address

Bit packing: Coil at starting address is in bit 0 (LSB) of first byte.

Example: Read 19 coils starting at address 19

Request PDU:

```
01 00 13 00 13
|  |     |
|  |     +-- Quantity: 19 coils (0x0013)
```

```
|  +-------- Start address: 19 (0x0013)
+----------- Function code: 0x01
```

Response PDU:

```
01 03 CD 6B 05
|  |  |  |  |
|  |  |  |  +-- Coils 35-37: 0x05 = 00000101 (37=ON, 36=OFF, 35=ON)
|  |  |  +----- Coils 27-34: 0x6B = 01101011
|  |  +-------- Coils 19-26: 0xCD = 11001101 (19=ON, 20=OFF, 21=ON, 22=ON,
...)
|  +----------- Byte count: 3 bytes
+-------------- Function code: 0x01
```

5.7 Function Code 0x05: Write Single Coil

Request (5 bytes)

Offset	Field	Size	Value
0	Function Code	1	0x05
1-2	Coil Address	2	0x0000 - 0xFFFF
3-4	Value	2	0xFF00 = ON, 0x0000 = OFF

Note: Only 0xFF00 and 0x0000 are valid. Other values are errors.

Response (5 bytes)

Echo of request.

Example: Turn ON coil 172

Request PDU:

```
05 00 AC FF 00
|  |     |
|  |     +-- Value: ON (0xFF00)
|  +-------- Address: 172 (0x00AC)
+----------- Function code: 0x05
```

5.8 Function Code 0x0F: Write Multiple Coils

Request (6 + N bytes)

Offset	Field	Size	Value Range
0	Function Code	1	0x0F
1-2	Starting Address	2	0x0000 - 0xFFFF
3-4	Quantity of Coils	2	0x0001 - 0x07B0 (1-1968)
5	Byte Count	1	⌈Quantity / 8⌉
6+	Coil Values	N	Packed bits, LSB = lowest address

Response (5 bytes)

Offset	Field	Size	Description
0	Function Code	1	0x0F
1-2	Starting Address	2	Address of first coil
3-4	Quantity of Coils	2	Number of coils written

5.9 Function Code 0x02: Read Discrete Inputs

Identical format to 0x01, but reads from Discrete Inputs table.

---

6. Exception Handling

Exception Response Format

When a server cannot process a request, it returns an exception response:

```
[Function Code + 0x80: 1 byte][Exception Code: 1 byte]
```

Example: Request function 0x03 fails → Response function = 0x83

Exception Codes

Code	Hex	Name	Description
01	0x01	ILLEGAL FUNCTION	Function code not supported by device
02	0x02	ILLEGAL DATA ADDRESS	Address not valid (out of range)

Code	Hex	Name	Description
03	0x03	ILLEGAL DATA VALUE	Value not valid for request
04	0x04	SERVER DEVICE FAILURE	Unrecoverable error during execution
05	0x05	ACKNOWLEDGE	Request accepted, long processing time
06	0x06	SERVER DEVICE BUSY	Device busy; client should retry
08	0x08	MEMORY PARITY ERROR	Extended file area parity check failed
10	0x0A	GATEWAY PATH UNAVAILABLE	Gateway misconfigured or overloaded
11	0x0B	GATEWAY TARGET NO RESPONSE	Target device failed to respond

Exception Example

Request to read register 9999 when device only has 100 registers:

Request PDU:

```
03 27 0F 00 01  (Read 1 register at address 9999)
```

Exception Response PDU:

```
83 02
|  |
|  +-- Exception code: 0x02 (ILLEGAL DATA ADDRESS)
+----- Function code: 0x03 + 0x80 = 0x83
```

---

7. Implementation Checklists

7.1 TCP Client Implementation

1. **Connection Setup**
   - [ ] Create TCP socket
   - [ ] Connect to server IP:502
   - [ ] Set TCP_NODELAY socket option
   - [ ] Set SO_KEEPALIVE socket option
   - [ ] Implement connection timeout
2. **Request Building**

- [ ] Generate unique Transaction ID (increment counter or random)
- [ ] Set Protocol ID = 0x0000
- [ ] Calculate Length = Unit ID (1) + PDU length
- [ ] Set Unit ID (0xFF for direct, 1-247 for gateway)
- [ ] Build PDU with function code and parameters
- [ ] All multi-byte values in big-endian

3. **Request/Response Cycle**
   - [ ] Send complete ADU in single TCP write
   - [ ] Implement response timeout (e.g., 1-5 seconds)
   - [ ] Read MBAP header first (7 bytes)
   - [ ] Validate Protocol ID = 0x0000
   - [ ] Read remaining bytes per Length field
   - [ ] Match Transaction ID to pending request
   - [ ] Check if function code has bit 7 set (exception)
4. **Error Handling**
   - [ ] Handle TCP connection errors
   - [ ] Handle response timeout
   - [ ] Parse exception responses
   - [ ] Implement retry logic (application-defined)

7.2 TCP Server Implementation

1. **Connection Setup**
   - [ ] Create TCP socket
   - [ ] Bind to port 502
   - [ ] Listen for connections
   - [ ] Accept multiple clients (connection pool)
   - [ ] Track connections for cleanup
2. **Request Processing**
   - [ ] Read MBAP header (7 bytes)
   - [ ] Validate Protocol ID = 0x0000
   - [ ] Read PDU per Length field
   - [ ] Extract function code
   - [ ] Validate addresses and values
   - [ ] Execute function or generate exception
3. **Response Building**
   - [ ] Copy Transaction ID from request
   - [ ] Set Protocol ID = 0x0000

- [ ] Calculate Length for response
- [ ] Copy Unit ID from request
- [ ] Build response PDU or exception

7.3 Common Pitfalls

Pitfall	Solution
Using 1-based addresses	Always use 0-based in PDU; convert from documentation
Wrong byte order	Use big-endian for all multi-byte fields
CRC byte order (RTU)	CRC is little-endian (LSB first), unlike all other fields
Incorrect Length field	Length = bytes after Length field (Unit ID + PDU)
Batch requests per TCP frame	Send one ADU per TCP send; don't concatenate
Ignoring Transaction ID	Must match response to request in async scenarios
Coil value encoding	Only 0xFF00 (ON) and 0x0000 (OFF) are valid for 0x05

---

8. Quick Reference

MBAP Header Template (TCP)

```
Byte 0-1:  Transaction ID     [XX XX]  Client-assigned
Byte 2-3:  Protocol ID        [00 00]  Always 0x0000
Byte 4-5:  Length             [XX XX]  Bytes following (Unit ID + PDU)
Byte 6:    Unit ID            [XX]     0xFF direct, 1-247 gateway
```

Function Code Quick Reference

Operation	Code	Request Data	Response Data
Read Coils	0x01	Addr(2) + Qty(2)	ByteCnt(1) + Bits(N)
Read Discrete Inputs	0x02	Addr(2) + Qty(2)	ByteCnt(1) + Bits(N)
Read Holding Registers	0x03	Addr(2) + Qty(2)	ByteCnt(1) + Regs(2N)
Read Input Registers	0x04	Addr(2) + Qty(2)	ByteCnt(1) + Regs(2N)
Write Single Coil	0x05	Addr(2) + Value(2)	Echo

Operation	Code	Request Data	Response Data
Write Single Register	0x06	Addr(2) + Value(2)	Echo
Write Multiple Coils	0x0F	Addr(2) + Qty(2) + ByteCnt(1) + Bits(N)	Addr(2) + Qty(2)
Write Multiple Registers	0x10	Addr(2) + Qty(2) + ByteCnt(1) + Regs(2N)	Addr(2) + Qty(2)

Exception Code Quick Reference

Code	Name	Typical Cause
0x01	ILLEGAL FUNCTION	Unsupported function code
0x02	ILLEGAL DATA ADDRESS	Address out of range
0x03	ILLEGAL DATA VALUE	Invalid quantity or value
0x04	SERVER DEVICE FAILURE	Internal device error
0x06	SERVER DEVICE BUSY	Retry later

---

9. Complete TCP Transaction Example

Scenario: Read 2 holding registers starting at address 0 from device at Unit ID 1.

Request

```
TCP ADU (12 bytes):
00 01 00 00 00 06 01 03 00 00 00 02
|     |     |     |  |  |     |
|     |     |     |  |  |     +-- Quantity: 2 registers
|     |     |     |  |  +-------- Start Address: 0
|     |     |     |  +----------- Function: 0x03 (Read Holding Registers)
|     |     |     +-------------- Unit ID: 1
|     |     +-------------------- Length: 6 bytes follow
|     +-------------------------- Protocol ID: 0 (MODBUS)
+-------------------------------- Transaction ID: 1
```

Response (Success)

```
TCP ADU (13 bytes):
00 01 00 00 00 07 01 03 04 00 0A 00 14
```

```
|     |     |     |  |  |  |     |
|     |     |     |  |  |  |     +-- Register 1 value: 0x0014 (20)
|     |     |     |  |  |  +-------- Register 0 value: 0x000A (10)
|     |     |     |  |  +----------- Byte count: 4 bytes
|     |     |     |  +-------------- Function: 0x03
|     |     |     +----------------- Unit ID: 1
|     |     +----------------------- Length: 7 bytes follow
|     +------------------------------ Protocol ID: 0
+----------------------------------- Transaction ID: 1 (matches request)
```

Response (Error)

```
TCP ADU (9 bytes):
00 01 00 00 00 03 01 83 02
|     |     |     |  |  |
|     |     |     |  |  +-- Exception Code: 0x02 (ILLEGAL DATA ADDRESS)
|     |     |     |  +----- Function: 0x83 (0x03 + 0x80)
|     |     |     +-------- Unit ID: 1
|     |     +-------------- Length: 3 bytes follow
|     +-------------------- Protocol ID: 0
+-------------------------- Transaction ID: 1
```

---

10. Serial Line Details

This section provides complete implementation details for MODBUS over serial lines (RTU and ASCII modes).

10.1 Master/Slave Protocol

MODBUS Serial Line is a **master-slave** protocol:

- One master, up to 247 slaves (addresses 1-247)
- Master initiates all transactions
- Slaves never communicate with each other
- Address 0 is reserved for broadcast (no response)

Mode	Description
Unicast	Master → specific slave → slave responds
Broadcast	Master → all slaves (address 0) → no response

10.2 MODBUS RTU Mode

RTU (Remote Terminal Unit) mode transmits data as binary with CRC-16 error checking.

10.2.1 RTU Character Format

Field	Bits	Description
Start bit	1	Always 0
Data bits	8	LSB first
Parity bit	1	Even parity (default), odd, or none
Stop bit(s)	1-2	1 with parity, 2 without parity
Total	**11**	Bits per character

Default configuration: 8 data bits, even parity, 1 stop bit (8E1)

Alternative (no parity): 8 data bits, no parity, 2 stop bits (8N2)

10.2.2 RTU Frame Structure

```
[Address: 1][Function: 1][Data: 0-252][CRC-16: 2]
|<- Slave ->|<-------- PDU ---------->|<- Check ->|
```

Maximum RTU ADU: 256 bytes (1 address + 253 PDU + 2 CRC)

10.2.3 RTU Frame Timing

Frames are delimited by silent intervals (no characters transmitted):

Timer	Duration	Purpose
t1.5	1.5 character times	Max gap between characters within frame
t3.5	3.5 character times	Min gap between frames

Timing calculation formula:

```
Character time (seconds) = bits_per_char / baud_rate
                         = 11 / baud_rate

t1.5 = 1.5 × character_time
t3.5 = 3.5 × character_time
```

Timing examples:

Baud Rate	Char Time	t1.5	t3.5
9600	1.146 ms	1.72 ms	4.01 ms
19200	0.573 ms	0.86 ms	2.01 ms
38400	0.286 ms	0.43 ms	1.00 ms
115200	0.095 ms	0.14 ms	0.33 ms

IMPORTANT: For baud rates > 19200 bps, use fixed timing values:

- t1.5 = 750 µs (inter-character timeout)
- t3.5 = 1.75 ms (inter-frame delay)

This simplifies implementation at high baud rates where precise timing becomes impractical.

10.2.4 RTU State Machine

```
                              t3.5 expired
          ┌──────────────────────────────────────┐
          │                                      ▼
     ┌─────────┐                           ┌─────────┐
     │ Initial │── t3.5 expired ──────────►│   Idle  │
     └─────────┘                           └─────────┘
                                                 │
                      ┌──────────────────────────┼──────────────────────────┐
                      │                          │                          │
                      ▼                          ▼                          ▼
                ┌──────────┐             ┌──────────┐
┌──────────┐
                │ Reception │             │  Emission  │             │  Control
│
                └──────────┘             └──────────┘             │ &
Waiting  │
                      │                          │
└──────────┘
                      │ t1.5 expired             │ last char                │
                      │                          │ + t3.5                   │
                      └──────────────────────────┼──────────────────────────┘
                                                 │
                                           t3.5 expired
                                                 │
                                                 ▼
                                            ┌──────────┐
```

```
                                                  │   Idle   │
                                                  └──────────┘
```

Frame reception rules:

1. Frame starts after t3.5 silence followed by first character
2. Characters must arrive within t1.5 of each other
3. If t1.5 exceeded between characters → frame error, discard
4. Frame ends when t3.5 silence detected after last character

10.2.5 CRC-16 Algorithm

Parameter	Value
Polynomial	0x8005 (bit-reversed: 0xA001)
Initial value	0xFFFF
Input reflection	Yes (LSB first)
Output reflection	Yes
Final XOR	0x0000

CRC byte order: LSB first (little-endian) - opposite of all other MODBUS fields

CRC-16 Pseudocode (bit-by-bit):

```
function crc16_modbus(data: bytes) -> uint16:
    crc = 0xFFFF
    for byte in data:
        crc = crc XOR byte
        for i in 0..7:
            if (crc AND 0x0001) != 0:
                crc = (crc >> 1) XOR 0xA001
            else:
                crc = crc >> 1
    return crc  // Transmit LSB first, then MSB
```

CRC-16 C Implementation (Lookup Table - Recommended):

```
/* High-order byte table */
static const unsigned char auchCRCHi[] = {
    0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81,
    0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81,
```

```
0x40, 0x01, 0xC0,
    0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1,
0x81, 0x40, 0x01,
    0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01,
0xC0, 0x80, 0x41,
    0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
0x00, 0xC1, 0x81,
    0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80,
0x41, 0x01, 0xC0,
    0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
0x80, 0x41, 0x01,
    0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00,
0xC1, 0x81, 0x40,
    0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81,
    0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81,
0x40, 0x01, 0xC0,
    0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1,
0x81, 0x40, 0x01,
    0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01,
0xC0, 0x80, 0x41,
    0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81,
    0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81,
0x40, 0x01, 0xC0,
    0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
0x80, 0x41, 0x01,
    0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01,
0xC0, 0x80, 0x41,
    0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81,
    0x40
};

/* Low-order byte table */
static const unsigned char auchCRCLo[] = {
    0x00, 0xC0, 0xC1, 0x01, 0xC3, 0x03, 0x02, 0xC2, 0xC6, 0x06, 0x07, 0xC7,
0x05, 0xC5, 0xC4,
    0x04, 0xCC, 0x0C, 0x0D, 0xCD, 0x0F, 0xCF, 0xCE, 0x0E, 0x0A, 0xCA, 0xCB,
0x0B, 0xC9, 0x09,
    0x08, 0xC8, 0xD8, 0x18, 0x19, 0xD9, 0x1B, 0xDB, 0xDA, 0x1A, 0x1E, 0xDE,
0xDF, 0x1F, 0xDD,
    0x1D, 0x1C, 0xDC, 0x14, 0xD4, 0xD5, 0x15, 0xD7, 0x17, 0x16, 0xD6, 0xD2,
0x12, 0x13, 0xD3,
    0x11, 0xD1, 0xD0, 0x10, 0xF0, 0x30, 0x31, 0xF1, 0x33, 0xF3, 0xF2, 0x32,
0x36, 0xF6, 0xF7,
```

```
    0x37, 0xF5, 0x35, 0x34, 0xF4, 0x3C, 0xFC, 0xFD, 0x3D, 0xFF, 0x3F, 0x3E,
0xFE, 0xFA, 0x3A,
    0x3B, 0xFB, 0x39, 0xF9, 0xF8, 0x38, 0x28, 0xE8, 0xE9, 0x29, 0xEB, 0x2B,
0x2A, 0xEA, 0xEE,
    0x2E, 0x2F, 0xEF, 0x2D, 0xED, 0xEC, 0x2C, 0xE4, 0x24, 0x25, 0xE5, 0x27,
0xE7, 0xE6, 0x26,
    0x22, 0xE2, 0xE3, 0x23, 0xE1, 0x21, 0x20, 0xE0, 0xA0, 0x60, 0x61, 0xA1,
0x63, 0xA3, 0xA2,
    0x62, 0x66, 0xA6, 0xA7, 0x67, 0xA5, 0x65, 0x64, 0xA4, 0x6C, 0xAC, 0xAD,
0x6D, 0xAF, 0x6F,
    0x6E, 0xAE, 0xAA, 0x6A, 0x6B, 0xAB, 0x69, 0xA9, 0xA8, 0x68, 0x78, 0xB8,
0xB9, 0x79, 0xBB,
    0x7B, 0x7A, 0xBA, 0xBE, 0x7E, 0x7F, 0xBF, 0x7D, 0xBD, 0xBC, 0x7C, 0xB4,
0x74, 0x75, 0xB5,
    0x77, 0xB7, 0xB6, 0x76, 0x72, 0xB2, 0xB3, 0x73, 0xB1, 0x71, 0x70, 0xB0,
0x50, 0x90, 0x91,
    0x51, 0x93, 0x53, 0x52, 0x92, 0x96, 0x56, 0x57, 0x97, 0x55, 0x95, 0x94,
0x54, 0x9C, 0x5C,
    0x5D, 0x9D, 0x5F, 0x9F, 0x9E, 0x5E, 0x5A, 0x9A, 0x9B, 0x5B, 0x99, 0x59,
0x58, 0x98, 0x88,
    0x48, 0x49, 0x89, 0x4B, 0x8B, 0x8A, 0x4A, 0x4E, 0x8E, 0x8F, 0x4F, 0x8D,
0x4D, 0x4C, 0x8C,
    0x44, 0x84, 0x85, 0x45, 0x87, 0x47, 0x46, 0x86, 0x82, 0x42, 0x43, 0x83,
0x41, 0x81, 0x80,
    0x40
};

/**
 * Calculate CRC-16 for MODBUS RTU
 * @param puchMsg   Pointer to message buffer
 * @param usDataLen Number of bytes in message
 * @return CRC value (already in correct byte order for transmission)
 */
unsigned short CRC16(unsigned char *puchMsg, unsigned short usDataLen)
{
    unsigned char uchCRCHi = 0xFF;  /* high byte of CRC initialized */
    unsigned char uchCRCLo = 0xFF;  /* low byte of CRC initialized */
    unsigned uIndex;

    while (usDataLen--) {
        uIndex = uchCRCLo ^ *puchMsg++;
        uchCRCLo = uchCRCHi ^ auchCRCHi[uIndex];
        uchCRCHi = auchCRCLo[uIndex];
    }
```

```
    return (uchCRCHi << 8 | uchCRCLo);
}
```

CRC Example:

```
Frame data: [01] [03] [00] [00] [00] [0A]  (Read 10 registers from slave 1)
CRC-16:     0xC5CD
Transmitted: [01] [03] [00] [00] [00] [0A] [CD] [C5]
                                           LSB  MSB
```

10.3 MODBUS ASCII Mode

ASCII mode transmits each byte as two ASCII hexadecimal characters. Less efficient but simpler timing requirements.

10.3.1 ASCII Character Format

Field	Bits	Description
Start bit	1	Always 0
Data bits	7	ASCII character, LSB first
Parity bit	1	Even parity (default)
Stop bit(s)	1-2	1 with parity, 2 without
Total	**10**	Bits per character

10.3.2 ASCII Frame Structure

```
[':'][Address: 2 chars][Function: 2 chars][Data: 2N chars][LRC: 2 chars][CR]
[LF]
 0x3A                                                                  0x0D
0x0A
```

Field	Value
Start delimiter	':' (0x3A)
End delimiter	CR LF (0x0D 0x0A)
Data encoding	Each byte → 2 ASCII hex chars ('0'-'9', 'A'-'F')
Max frame size	513 characters

Maximum ASCII ADU: 513 chars (1 start + 2×(1+1+252+1) + 2 end)

10.3.3 ASCII Timing

- **Inter-character timeout:** Up to 1 second between characters (configurable)
- **Frame delimiter:** Receiving ':' starts new frame (any partial frame discarded)

10.3.4 LRC Calculation

LRC (Longitudinal Redundancy Check) is the two's complement of the 8-bit sum of all bytes.

LRC Calculation:

1. Sum all bytes (address through last data byte)
2. Discard carries (keep only lower 8 bits)
3. Take two's complement (negate)

LRC C Implementation:

```c
/**
 * Calculate LRC for MODBUS ASCII
 * @param auchMsg   Pointer to message buffer (binary, not ASCII-encoded)
 * @param usDataLen Number of bytes in message
 * @return LRC value (8-bit)
 */
static unsigned char LRC(unsigned char *auchMsg, unsigned short usDataLen)
{
    unsigned char uchLRC = 0;

    while (usDataLen--) {
        uchLRC += *auchMsg++;
    }

    return ((unsigned char)(-((char)uchLRC)));
}
```

LRC Example:

```
Binary data:   [01] [03] [00] [00] [00] [0A]
Sum:           0x01 + 0x03 + 0x00 + 0x00 + 0x00 + 0x0A = 0x0E
LRC:           -0x0E = 0xF2

ASCII frame:   :0103000000F2<CR><LF>
               | |  |    | |
               | |  |    | +-- LRC "F2"
```

```
                | |  |    +---- Quantity "000A" → "00" "0A"
                | |  +--------- Start address "0000"
                | +------------ Function "03"
                +-------------- Slave address "01"
```

10.4 Physical Layer (RS-485)

10.4.1 Two-Wire RS-485 Configuration

The standard MODBUS serial configuration uses RS-485 2-wire (half-duplex).

Signal	Description
D1 (B/B')	Data line 1, V1 voltage (V1 > V0 for binary 1/OFF)
D0 (A/A')	Data line 0, V0 voltage (V0 > V1 for binary 0/ON)
Common (C)	Signal and power supply common

10.4.2 Bus Specifications

Parameter	Value
Max devices	32 (without repeater)
Max cable length	1000m at 9600 baud (AWG24 or wider)
Max derivation length	20m
Baud rates (required)	9600, 19200 (default: 19200)
Baud rates (optional)	1200, 2400, 4800, 38400, 56K, 115K
Baud rate tolerance	±1% transmit, ±2% receive

10.4.3 Line Termination

- Required at **both ends** of trunk cable
- **Termination resistor:** 150Ω (0.5W) between D0 and D1
- Alternative: 120Ω + 1nF capacitor in series (for biased lines)

10.4.4 Line Polarization (Biasing)

When bus is idle (no transmission), receivers may need bias to prevent noise triggering:

Resistor	Connection	Value
Pull-up	D1 to +5V	450-650Ω
Pull-down	D0 to Common	450-650Ω

Only implement at **one location** (typically master/tap). Using 650Ω allows more devices on bus.

10.5 Serial Implementation Checklist

10.5.1 RTU Master Implementation

1. **Serial Port Setup**
   - [ ] Configure baud rate (9600 or 19200 default)
   - [ ] Set 8 data bits, even parity, 1 stop bit (8E1)
   - [ ] Calculate t1.5 and t3.5 timers for baud rate
   - [ ] Use fixed timers (750µs, 1.75ms) if baud > 19200
2. **Transmission**
   - [ ] Wait for t3.5 silence before sending
   - [ ] Build frame: [Address][Function][Data][CRC-Lo][CRC-Hi]
   - [ ] Transmit all bytes as continuous stream
   - [ ] Start response timeout after transmission
3. **Reception**
   - [ ] Detect frame start after t3.5 silence
   - [ ] Accumulate bytes, verify t1.5 inter-character timing
   - [ ] Detect frame end on t3.5 silence
   - [ ] Validate CRC-16
   - [ ] Verify response address matches request
4. **Timeouts**
   - [ ] Response timeout: 1-5 seconds typical
   - [ ] Turnaround delay (broadcast): 100-200ms
   - [ ] Implement retry logic (application-specific)

10.5.2 RTU Slave Implementation

1. **Reception**
   - [ ] Detect frame start after t3.5 silence
   - [ ] Validate t1.5 inter-character timing
   - [ ] Check CRC-16
   - [ ] Verify address (own address or broadcast 0)

- [ ] Discard frames with errors silently (no response)

2. **Processing**
   - [ ] Parse function code and data
   - [ ] Execute requested operation or generate exception
   - [ ] For broadcast: execute but don't respond
3. **Transmission**
   - [ ] Build response: [Address][Function][Data][CRC-Lo][CRC-Hi]
   - [ ] Wait for turnaround time if needed
   - [ ] Transmit all bytes as continuous stream

10.6 Complete RTU Transaction Example

Scenario: Master reads 2 holding registers starting at address 0 from slave 1 at 9600 baud.

Timing Diagram

```
Master TX:  ═══[01][03][00][00][00][02][C4][0B]═══
                 |   |   |   |   |   |   |   |
                 |   |   |   |   |   |   └───┴── CRC (0x0BC4, LSB first)
                 |   |   |   |   └───┴──────────── Quantity: 2
                 |   |   └───┴──────────────────── Start address: 0
                 |   └──────────────────────────── Function: Read Holding
Registers
                 └──────────────────────────────── Slave address: 1

            ←──── 4.01ms ────→                 ←── 4.01ms ──→
Silent:     ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒               ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒
            (t3.5 before TX)                   (t3.5 before RX)

Slave RX:   ═══[01][03][00][00][00][02][C4][0B]═══
            (processes request)

Slave TX:                                      ═══[01][03][04][00][0A][00]
[14][xx][xx]═══
                                                    |   |   |   |   |   |
|   |   |
                                                    |   |   |   |   |   |
└───┴── CRC
                                                    |   |   |   |
└───┴──────────── Reg 1: 20
                                                    |   |   |
└──────────────── Reg 0: 10
                                                    |   |
└──────────────────── Byte count: 4
```

```
                                                    │
└──────────────────────────── Function: 0x03

└──────────────────────────────── Address: 1
```

Frame Bytes (Hex)

Request (8 bytes):

```
01 03 00 00 00 02 C4 0B
```

Response (9 bytes):

```
01 03 04 00 0A 00 14 [CRC-Lo] [CRC-Hi]
```

---

11. Implementation Notes for Solar Inverters

When implementing MODBUS for solar inverter communication:

Common Register Mappings (Vendor-Specific)

Data Type	Typical Registers	Access
DC Voltage	Input registers (0x04)	Read-only
DC Current	Input registers (0x04)	Read-only
AC Power	Input registers (0x04)	Read-only
Total Energy	Holding registers (0x03)	Read-only
Power Limit	Holding registers (0x06, 0x10)	Read/Write
Device Status	Discrete inputs (0x02)	Read-only

Best Practices

1. **Always consult vendor documentation** for register maps
2. **Use appropriate polling intervals** (1-10 seconds typical)
3. **Handle 32-bit values correctly** - verify word order (big-endian vs little-endian registers)
4. **Implement robust error handling** - solar inverters may go offline at night
5. **Consider RS-485 cable length** - solar installations often span large distances

Document version: 1.1
Generated for AI implementation assistance
Source: MODBUS Application Protocol Specification V1.1b3, MODBUS Messaging Implementation Guide V1.0b, MODBUS/TCP Security Protocol V36, MODBUS over Serial Line Specification V1.02